Analytical Model Module Specification (v2.1)

Unified Theoretical Framework & Implementation in `analytic_model.py`

Manuscript: "Graphene-Assisted Electrostatic Interface Engineering for Radiation-Tolerant MR-DWELL Dosimeters"
Authors: Bigul Dosymova, Mikhail Dolgopolov
Revision: Radiation Measurements Revision (v2.1)

1. Executive Overview & Design Principles

The `analytic_model.py` module serves as the **single source of truth** for all theoretical and numerical calculations across the manuscript and supplementary material. By encapsulating physical parameters, electrostatic formulas, and device performance models into a unified API, it guarantees complete consistency between analytical figures (Figs. 2–8) and classification routines.

- **Decoupled Architecture:** Physical physics calculations are strictly separated from Matplotlib rendering scripts.
- **Decoupled Sensitivity & Saturation:** Low-dose sensitivity (α_{eff}), characteristic dose (D_0), and asymptotic shift (ΔE_{max}) are treated as independent phenomenological parameters ($\Delta E_{max} \neq \alpha_{eff} \cdot D_0$ in the general case), eliminating physical inconsistencies identified in previous revisions.
- **Immutable & Cached Parameters:** Implemented via Python's `@dataclass(frozen=True)` with eager initialization caching for high-performance array evaluations.

2. Mathematical & Electrostatic Formulation

2.1 Electrostatic Screening & Quantum Capacitance

The active tunneling region consists of a geometrical capacitance C_{geo} in series with the graphene quantum capacitance C_Q:

$$C_{geo} = \varepsilon_0 \, \varepsilon_r \, A \, / \, t_d, \qquad C_Q = (C_Q \, / \, A) \cdot A$$

The quantum-capacitance ratio η and electrostatic screening factor S are defined as (Eqs. 3–4):

$$\eta = C_Q \, / \, C_{geo}, \qquad S = \eta \, / \, (1 + \eta) = C_Q \, / \, (C_Q + C_{geo})$$

2.2 Radiation Trap Response & Resonance Shifts

The total radiation-induced interface trapped charge $Q_{tr}(D)$ follows a universal exponential saturation function $f(D)$ (Eq. 9):

$$f(D) = 1 - \exp(-D \, / \, D_0), \qquad Q_{tr}(D) = Q_{max} \cdot f(D), \qquad Q_{max} = q \cdot D_{it} \cdot \Delta E_{gap} \cdot A$$

The resonance level energy $E_i(D)$ and linewidth $\Gamma_i(D)$ for levels $i \in \{1, 2\}$ shift according to (Eqs. 10–11):

$$E_i(D) = E_{i0} - \Delta E_{i,max} \cdot [1 - \exp(-D \, / \, D_0)]$$

$$\Gamma_i(D) = \Gamma_{i0} + \Delta \Gamma_{i,max} \cdot [1 - \exp(-D \, / \, D_0)]$$

2.3 Device Performance Scaling (Figure 6 Models)

The Peak-to-Valley Ratio (PVR) and normalized peak current (I_{peak}^{norm}) scale with capacitance ratio η:

$$PVR(\eta) = PVR_{metal} + (PVR_{graphene} - PVR_{metal}) \cdot [1 - \exp(-\lambda_{PVR}\ \eta)]$$

$$I_{peak}^{norm}(\eta) = I_{metal}^{norm} + (I_{\infty}^{norm} - I_{metal}^{norm}) \cdot [1 - \exp(-\lambda_{I}\ \eta)]$$

3. Complete Parameter Reference Table

Parameter	Python Attribute	Default Value	Physical Units	Description
T	`temperature_K`	300.0	K	Operating device temperature
A	`emitter_area_cm2`	1.0e-10	cm^2	Active emitter tunnel area
ε_r	`dielectric_constant`	3.9	dimensionless	Relative dielectric constant (h-BN)
t_d	`dielectric_thickness_nm`	1.0	nm	Dielectric spacer thickness
C_Q / A	`CQ_per_area_uF_cm2`	7.2	$\mu F/cm^2$	Graphene quantum capacitance per area
E_{10}, E_{20}	`E10_meV, E20_meV`	82.0, 126.0	meV	Unirradiated resonance energy levels
Γ_{10}, Γ_{20}	`Gamma10_meV, Gamma20_meV`	4.0, 7.0	meV	Unirradiated resonance linewidths
$\alpha_{1,eff}, \alpha_{2,eff}$	`alpha1_eff_meV_per_mGy, ...`	0.045, 0.020	meV/mGy	Low-dose sensitivity slopes
D_0	`D0_Gy`	500.0	Gy	Characteristic saturation dose
$\Delta E_{1,max}, \Delta E_{2,max}$	`DeltaE1_max_meV, ...`	22.5, 10.0	meV	Asymptotic maximum resonance energy shifts
$\Delta\Gamma_{1,max}, \Delta\Gamma_{2,max}$	`DeltaGamma1_max_meV, ...`	0.40, 0.25	meV	Maximum linewidth broadening
D_{it}	`Dit_cm2_eV`	1.0e11	$cm^{-2}\ eV^{-1}$	Interface state density

4. API Structure & Code Integration Example

The module provides helper factory functions `create_default_parameters()` and `create_reference_device(expected_eta=2.09)` that instantiate validated parameters.

```python
import numpy as np
from analytic_model import create_reference_device, validate_parameters

# 1. Initialize reference device calibrated to manuscript eta = 2.09
params = create_reference_device(expected_eta=2.09)

# 2. Compute resonance energy shift for a vector of doses
doses = np.array([0.0, 100.0, 500.0, 1000.0])  # Dose in Gy
delta_E1 = params.resonance_shift(doses, level=1)  # meV

# 3. Retrieve screening factor and PVR performance
S = params.screening_factor()             # S = 0.676
pvr_val = params.pvr()                      # PVR ≈ 10.0
```

Validation Suite

Executing `params.validate()` runs multi-tier physical checks (e.g. verifying $T > 0$, $E_{20} > E_{10}$, $0 \leq S \leq 1$, and capacitance consistency) and throws a descriptive `ParameterError` if any parameter violates physical bounds.